

# Spring Boot

---

CatMono

April 15, 2026

# 目 录

---

|                  |                       |           |
|------------------|-----------------------|-----------|
| <b>I</b>         | <b>Spring Boot 基础</b> |           |
| <b>1 第01章</b>    |                       | <b>4</b>  |
| 1 占位内容 . . . . . |                       | 4         |
| <b>2 第02章</b>    |                       | <b>5</b>  |
| 1 占位内容 . . . . . |                       | 5         |
| <b>3 第03章</b>    |                       | <b>6</b>  |
| 1 占位内容 . . . . . |                       | 6         |
| <b>4 第04章</b>    |                       | <b>7</b>  |
| 1 占位内容 . . . . . |                       | 7         |
| <b>II</b>        | <b>Spring Boot 核心</b> |           |
| <b>5 第05章</b>    |                       | <b>9</b>  |
| 1 占位内容 . . . . . |                       | 9         |
| <b>6 第06章</b>    |                       | <b>10</b> |
| 1 占位内容 . . . . . |                       | 10        |
| <b>7 第07章</b>    |                       | <b>11</b> |
| 1 占位内容 . . . . . |                       | 11        |
| <b>8 第08章</b>    |                       | <b>12</b> |
| 1 占位内容 . . . . . |                       | 12        |
| <b>9 第09章</b>    |                       | <b>13</b> |
| 1 占位内容 . . . . . |                       | 13        |
| <b>10 第10章</b>   |                       | <b>14</b> |
| 1 占位内容 . . . . . |                       | 14        |

|                  |           |
|------------------|-----------|
| <b>11 第10章</b>   | <b>15</b> |
| 1 占位内容 . . . . . | 15        |

## III

## 微服务与中间件

|                                        |           |
|----------------------------------------|-----------|
| <b>12 微服务架构基础</b>                      | <b>17</b> |
| 1 微服务架构概述 . . . . .                    | 17        |
| 1.1 微服务架构简介 . . . . .                  | 17        |
| 1.2 Spring Cloud 与微服务生态 . . . . .      | 18        |
| 2 服务注册与发现 . . . . .                    | 20        |
| 2.1 核心组件 . . . . .                     | 20        |
| 2.2 主流实现 . . . . .                     | 20        |
| 2.3 Nacos 实践示例 . . . . .               | 21        |
| 3 配置中心 . . . . .                       | 24        |
| 3.1 主流实现 . . . . .                     | 24        |
| 3.2 Nacos 配置中心实践示例 . . . . .           | 25        |
| 4 API 网关 . . . . .                     | 28        |
| 4.1 SpringCloud Gateway . . . . .      | 29        |
| 4.2 SpringCloud Gateway 实践示例 . . . . . | 30        |
| 5 负载均衡与熔断降级 . . . . .                  | 31        |
| <b>13 第10章</b>                         | <b>32</b> |
| 1 占位内容 . . . . .                       | 32        |
| <b>14 第10章</b>                         | <b>33</b> |
| 1 占位内容 . . . . .                       | 33        |

## IV

## Spring Boot 源码

# I

## Part

# Spring Boot 基础

|          |                     |          |
|----------|---------------------|----------|
| <b>1</b> | <b>第 01 章 .....</b> | <b>4</b> |
| 1        | 占位内容 .....          | 4        |
| <b>2</b> | <b>第 02 章 .....</b> | <b>5</b> |
| 1        | 占位内容 .....          | 5        |
| <b>3</b> | <b>第 03 章 .....</b> | <b>6</b> |
| 1        | 占位内容 .....          | 6        |
| <b>4</b> | <b>第 04 章 .....</b> | <b>7</b> |
| 1        | 占位内容 .....          | 7        |

# Chapter 1

## 第 01 章

---

### 1 占位内容

这里是 chap01 的初始内容。

# Chapter 2

## 第 02 章

---

### 1 占位内容

这里是 chap02 的初始内容。

# Chapter 3

## 第 03 章

---

### 1 占位内容

这里是 chap03 的初始内容。

# Chapter 4

## 第 04 章

---

### 1 占位内容

这里是 chap04 的初始内容。



# II

Part

## Spring Boot 核心

|    |              |    |
|----|--------------|----|
| 5  | 第 05 章 ..... | 9  |
| 1  | 占位内容 .....   | 9  |
| 6  | 第 06 章 ..... | 10 |
| 1  | 占位内容 .....   | 10 |
| 7  | 第 07 章 ..... | 11 |
| 1  | 占位内容 .....   | 11 |
| 8  | 第 08 章 ..... | 12 |
| 1  | 占位内容 .....   | 12 |
| 9  | 第 09 章 ..... | 13 |
| 1  | 占位内容 .....   | 13 |
| 10 | 第 10 章 ..... | 14 |
| 1  | 占位内容 .....   | 14 |
| 11 | 第 10 章 ..... | 15 |
| 1  | 占位内容 .....   | 15 |

# Chapter 5

## 第 05 章

---

### 1 占位内容

这里是 chap05 的初始内容。

# Chapter 6

## 第 06 章

---

### 1 占位内容

这里是 chap06 的初始内容。

# Chapter 7

## 第 07 章

---

### 1 占位内容

这里是 chap07 的初始内容。

# Chapter 8

## 第 08 章

---

### 1 占位内容

这里是 chap08 的初始内容。

# Chapter 9

## 第 09 章

---

### 1 占位内容

这里是 chap09 的初始内容。

# Chapter 10

## 第 10 章

---

### 1 占位内容

这里是 chap10 的初始内容。

# Chapter 11

## 第 10 章

---

### 1 占位内容

这里是 chap10 的初始内容。



# Part III

## 微服务与中间件

|    |                 |    |
|----|-----------------|----|
| 12 | 微服务架构基础 .....   | 17 |
| 1  | 微服务架构概述 .....   | 17 |
| 2  | 服务注册与发现 .....   | 20 |
| 3  | 配置中心 .....      | 24 |
| 4  | API 网关 .....    | 28 |
| 5  | 负载均衡与熔断降级 ..... | 31 |
| 13 | 第 10 章 .....    | 32 |
| 1  | 占位内容 .....      | 32 |
| 14 | 第 10 章 .....    | 33 |
| 1  | 占位内容 .....      | 33 |

# Chapter 12

## 微服务架构基础

### 1 微服务架构概述

#### 1.1 微服务架构简介

**微服务 (Microservices)** 是一种软件架构风格, 它是以专注于单一责任与功能的小型功能区块 (Small Building Blocks) 为基础, 利用模块化的方式组合出复杂的大型应用程序, 各功能区块使用与语言无关 (Language Independent), 而且复杂的服务背后是使用简单 URI 来开放界面, 任何服务, 任何细粒都能被开放 (exposed). 这个设计在 HP 的实验室被实现, 具有改变复杂软件系统的强大力量.

2014 年, Martin Fowler 与 James Lewis 共同提出了微服务的概念, 定义了微服务是由以单一应用程序构成的小服务, 自己拥有自己的进程与轻量化处理, 服务依业务功能设计, 以全自动的方式部署, 与其他服务使用 HTTP API 通信. 同时服务会使用最小的规模的集中管理 (例如 Docker) 能力, 服务可以用不同的编程语言与数据库等组件实现.

核心思想: “分而治之”, 将单体应用按业务领域 (Domain-Driven Design, DDD) 拆分为多个松耦合的服务。核心特征:

**单一职责** 每个微服务专注于一个业务功能, 职责单一, 易于理解和维护。

**独立部署** 每个微服务可以独立构建、测试和部署, 减少了部署风险和复杂度。

**技术异构** 不同微服务可以使用不同的编程语言、数据库和技术栈, 适合不同的业务需求。

**轻量通信** 微服务之间通过轻量级协议 (如 HTTP/REST、gRPC) 进行通信, 降低了服务间的耦合度。

**自治性** 每个微服务拥有自己的数据库和数据管理方式, 避免了共享数据库带来的耦合问题。

**弹性与可伸缩性** 微服务可以根据需求独立伸缩, 提高系统的弹性和性能。

下表 (12.1) 是单体应用、SOA 和微服务架构的对比:

#### NOTE

ESB (Enterprise Service Bus) 是一种**重量级**的企业级服务总线, 提供了服务注册、发现、路由、协议转换等功能, 适用于 SOA 架构。

SOAP (Simple Object Access Protocol) 是一种基于 XML 的协议, 用于在网络上交换结构化信息, 常用于 SOA 架构中的服务通信。

gRPC (Google Remote Procedure Call) 是一种高性能、开源的远程过程调用框架, 支持多种编程语言, 常用于微服务架构中的服务通信。

微服务架构在带来灵活性、可伸缩性和快速迭代的同时, 也引入了分布式系统的复杂性, 如服务间通信、数据一致性、监控和调试、运维复杂性等挑战。因此, 选择微服务架构需要根据具体业务需求、团队能力和技术栈进行权衡。

Table 12.1: 单体应用、SOA 与微服务架构对比

| 对比维度 | 单体应用           | SOA                 | 微服务                       |
|------|----------------|---------------------|---------------------------|
| 服务粒度 | 粗粒度，模块集中在同一应用中 | 中等粒度，按企业能力划分服务      | 细粒度，按业务能力拆分为小服务           |
| 部署方式 | 整体打包与统一部署      | 服务可独立部署，但常依赖集中式平台   | 服务独立构建、独立部署、可独立扩缩容        |
| 通信方式 | 进程内调用为主        | 常见 ESB、SOAP 等较重通信方式 | 轻量协议（HTTP/REST、gRPC、消息队列） |
| 数据管理 | 通常共享一个数据库      | 可能存在共享数据模型          | 每个服务拥有自治数据，避免强耦合          |
| 技术栈  | 技术栈统一，升级牵一发动全身 | 相对统一，强调企业级标准化       | 技术异构，按场景选择最合适技术           |
| 适用场景 | 业务简单、团队小、快速起步  | 跨系统集成、存量系统较多的大型企业   | 业务复杂、迭代快、需要高可用与弹性扩展       |

## 1.2 Spring Cloud 与微服务生态

**Spring Cloud** 是 Pivotal 团队提供的开源微服务框架，是基于 Spring Boot 的一套工具集，它将 Netflix、Alibaba 等公司的成熟微服务组件整合到 Spring 生态中，帮助开发者快速构建微服务架构。Spring Cloud 提供了以下核心组件：

**服务注册与发现** Nacos、Consul、Eureka (已停止维护) 等组件提供了服务注册与发现功能，支持服务实例的动态注册和查询。Nacos 集成了注册与配置中心，成为事实标准。

**配置中心** Nacos Config、Spring Cloud Config 等组件提供了集中化的配置管理，支持动态刷新和版本控制。

**API 网关** Spring Cloud Gateway 提供了一个基于 Spring WebFlux 的非阻塞网关，支持路由、负载均衡、熔断、限流等功能。

**服务调用** Spring Cloud OpenFeign 提供了声明式的 REST 客户端，简化了服务间的调用和通信。

**负载均衡与熔断降级** Spring Cloud LoadBalancer 提供了客户端负载均衡功能，Resilience4j 提供了熔断和降级功能，帮助提高系统的稳定性和容错能力。

### INFO

Spring Cloud 2020.x 之后移除了 Netflix 组件（Ribbon、Hystrix、Zuul 1.x），转向 Spring Cloud LoadBalancer、Resilience4j、Spring Cloud Gateway。  
2025 年起，Nacos 与 Kubernetes 原生服务发现成为主流，Spring Cloud Kubernetes 提供了与 K8s 原生服务注册的集成。

Spring Cloud 的版本命名早期使用 London 地铁站名（如 Hoxton、Greenwich），2020 年后使用日历化版本（如 2020.0.x、2021.0.x）。其版本对应关系如表 (12.2, 12.3, 12.4) 所示。

### WARNING

Spring 官方说明：Dalston、Edgware、Finchley、Greenwich、2020.0、2021.0、2022.0 已结束支持（EOL）。

新项目开发建议使用 SpringBoot 3.2.4 + Spring Cloud 2023.0.1 + Spring Cloud Alibaba 2023.0.1.0 版本组合。SpringBoot 3.2.4 版本是 3.2 系列的稳定补丁版，配套 Spring Framework 6.1.x、Spring Security 6.2.x，最低支持 Java 17，推荐使用 Java 21。

Table 12.2: Spring Cloud Release Train 与 Spring Boot 对应关系 (官方)

| Release Train          | Spring Boot Generation                |
|------------------------|---------------------------------------|
| 2025.1.x (Oakwood)     | 4.0.x                                 |
| 2025.0.x (Northfields) | 3.5.x                                 |
| 2024.0.x (Moorgate)    | 3.4.x                                 |
| 2023.0.x (Leyton)      | 3.3.x, 3.2.x                          |
| 2022.0.x (Kilburn)     | 3.0.x, 3.1.x (Starting with 2022.0.3) |
| 2021.0.x (Jubilee)     | 2.6.x, 2.7.x (Starting with 2021.0.3) |
| 2020.0.x (Ilford)      | 2.4.x, 2.5.x (Starting with 2020.0.3) |
| Hoxton                 | 2.2.x, 2.3.x (Starting with SR5)      |
| Greenwich              | 2.1.x                                 |
| Finchley               | 2.0.x                                 |
| Edgware                | 1.5.x                                 |
| Dalston                | 1.5.x                                 |

Table 12.3: Spring Cloud Alibaba 2025.x 2021.x 版本对应关系 (官方)

| Spring Cloud Alibaba Version | Spring Cloud Version  | Spring Boot Version |
|------------------------------|-----------------------|---------------------|
| 2025.1.0.0                   | 2025.1.0              | 4.0.0               |
| 2025.0.0.0                   | 2025.0.0              | 3.5.0               |
| 2023.0.1.0                   | Spring Cloud 2023.0.1 | 3.2.4               |
| 2023.0.0.0-RC1               | Spring Cloud 2023.0.0 | 3.2.0               |
| 2022.0.0.0                   | Spring Cloud 2022.0.0 | 3.0.2               |
| 2022.0.0.0-RC2               | Spring Cloud 2022.0.0 | 3.0.2               |
| 2022.0.0.0-RC1               | Spring Cloud 2022.0.0 | 3.0.0               |
| 2021.0.6.0                   | Spring Cloud 2021.0.5 | 2.6.13              |
| 2021.0.5.0                   | Spring Cloud 2021.0.5 | 2.6.13              |
| 2021.0.4.0                   | Spring Cloud 2021.0.4 | 2.6.11              |
| 2021.0.1.0                   | Spring Cloud 2021.0.1 | 2.6.3               |
| 2021.1                       | Spring Cloud 2020.0.1 | 2.4.2               |

Table 12.4: Spring Cloud Alibaba 2.2.x 及以下版本对应关系 (官方)

| Spring Cloud Alibaba Version | Spring Cloud Version        | Spring Boot Version |
|------------------------------|-----------------------------|---------------------|
| 2.2.10-RC2*                  | Spring Cloud Hoxton.SR12    | 2.3.12.RELEASE      |
| 2.2.10-RC1                   | Spring Cloud Hoxton.SR12    | 2.3.12.RELEASE      |
| 2.2.9.RELEASE                | Spring Cloud Hoxton.SR12    | 2.3.12.RELEASE      |
| 2.2.8.RELEASE                | Spring Cloud Hoxton.SR12    | 2.3.12.RELEASE      |
| 2.2.7.RELEASE                | Spring Cloud Hoxton.SR12    | 2.3.12.RELEASE      |
| 2.2.6.RELEASE                | Spring Cloud Hoxton.SR9     | 2.3.2.RELEASE       |
| 2.2.1.RELEASE                | Spring Cloud Hoxton.SR3     | 2.2.5.RELEASE       |
| 2.2.0.RELEASE                | Spring Cloud Hoxton.RELEASE | 2.2.X.RELEASE       |
| 2.1.4.RELEASE                | Spring Cloud Greenwich.SR6  | 2.1.13.RELEASE      |
| 2.1.2.RELEASE                | Spring Cloud Greenwich      | 2.1.X.RELEASE       |
| 2.0.4.RELEASE (停止维护, 建议升级)   | Spring Cloud Finchley       | 2.0.X.RELEASE       |
| 1.5.1.RELEASE (停止维护, 建议升级)   | Spring Cloud Edgware        | 1.5.X.RELEASE       |

## 2 服务注册与发现

在微服务架构中，一个完整的应用被拆分成多个独立的服务。这些服务通常运行在不同的服务器上，拥有动态的 IP 地址和端口号，且服务实例的位置是动态变化的（扩缩容、故障重启、发布更新），尤其是在使用容器化技术（如 Docker）和云平台时。

这就带来了一个核心问题：服务 A 如何知道服务 B 的地址并与之通信？换句话说，**服务消费者如何找到服务提供者**？

- 传统单体应用：不同模块之间通过方法调用直接通信，地址是固定的。
- 微服务应用：服务地址是动态变化的。如果硬编码 IP 地址和端口，一旦服务重启、扩容或缩容，地址就会失效，导致整个系统瘫痪。

**服务注册与发现机制 (Service Registration and Discovery)** 就是为了解决这个问题而诞生的。它提供了一个**动态、自动化的方式来管理服务间的通信地址**。

### 2.1 核心组件

服务注册与发现模式主要包含三个角色：

1. 服务注册中心 (Service Registry): 一个集中式的服务目录，相当于一个“微服务通讯录”，负责维护所有可用服务实例的信息，包括服务名称、IP 地址、端口号等元数据。
2. 服务提供者 (Service Provider): 提供具体业务功能的服务实例，在启动时向服务注册中心**注册**自己的信息，并定期发送心跳 (Heartbeat) 以保持注册状态。
3. 服务消费者 (Service Consumer): 需要调用其他服务的服务实例，在调用时向服务注册中心**查询**目标服务的地址列表，注册中心返回一个或多个地址。消费者会选择其中一个（通常通过负载均衡策略）发起调用。

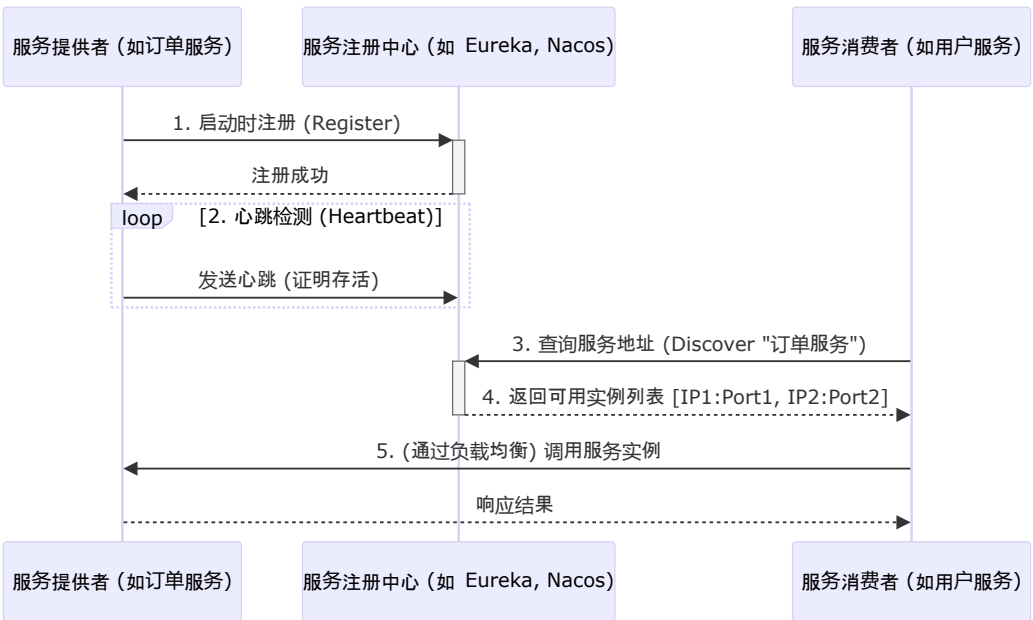


Figure 12.1: 服务注册与发现机制示意图

### 2.2 主流实现

目前主流的服务注册与发现组件包括 Eureka、Nacos、Consul 和 Zookeeper 等。它们各有特点，适用于不同的场景和需求：

**Eureka (Netflix)** 经典的服务注册中心，使用门槛低，易于上手。AP 架构（高可用优先），采用服务端与客户端协作模式，节点之间采用对等复制（P2P）机制。当前处于维护模式（Maintenance Mode），新项目一般不推荐作为首选，但在存量系统中仍较常见。

**Nacos (Alibaba)** 同时提供服务注册与配置管理能力，一体化程度高。支持 CP/AP 模式切换，具备服务权重、健康检查、元数据管理等能力。提供可视化控制台，运维体验较好。综合能力强、社区活跃，属于当前国内微服务体系中的主流推荐方案。

**Consul (HashiCorp)** 生态完整，除注册发现外还提供 KV 存储与多数据中心支持。以 CP 架构为主（强一致性优先），健康检查机制完善。适合对一致性要求较高，或已采用 HashiCorp 技术栈的团队。

**Zookeeper (Apache)** 本质是分布式协调服务，也可用于服务注册发现。CP 架构（强一致性），采用树形节点数据模型。方案相对传统、配置复杂度较高，在新微服务项目中通常不是首选。

## 2.3 Nacos 实践示例

下面以 Nacos 为例，介绍服务注册与发现的基本使用（更详细的教程可在[Nacos 官方文档](#)中找到）。

### 1. 搭建 Nacos 服务注册中心

推荐使用 Docker 快速搭建 Nacos 服务注册中心，默认 API 调用端口为 8848，控制台访问端口为 8080：

docker-compose.yml

```
1 services:
2   nacos:
3     image: nacos/nacos-server:latest
4     container_name: nacos
5     ports:
6       - "8848:8848" # Nacos API 端口
7       - "8080:8080" # Nacos 控制台端口
8     environment:
9       # 设置自定义认证TOKEN: SecretKey012345678901234567890123456789
10      - NACOS_AUTH_TOKEN=U2VjcmV0S2V5MDEyMzQ1Njc4OTAxMjM0NTY3ODkwMTIzNDU2Nzg5
11      # 设置身份标识（请确保这两个都已添加）
12      - NACOS_AUTH_IDENTITY_KEY=xxxxxx
13      - NACOS_AUTH_IDENTITY_VALUE=xxxxxxx
14      - MODE=standalone # 单机模式，适合开发测试
15      - SPRING_DATASOURCE_PLATFORM=mysql # 使用 MySQL 作为持久化存储
16      - MYSQL_SERVICE_HOST=mysql
17      - MYSQL_SERVICE_PORT=3306
18      - MYSQL_SERVICE_DB_NAME=nacos_config
19      - MYSQL_SERVICE_USER=root
20      - MYSQL_SERVICE_PASSWORD=123456
21     volumes:
22       - ./nacos-data:/home/nacos/data
23     depends_on:
24       - mysql
```

### 2. 配置服务提供者

在 pom.xml 中添加 Nacos Discovery 依赖（Gradle 同理）：

pom.xml

```
1 <dependency>
2   <groupId>com.alibaba.cloud</groupId>
3   <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
4 </dependency>
```



**TIP**

版本号由 Spring Cloud Alibaba BOM（物料清单）统一管理。在 `<dependencyManagement>` 中引入 `spring-cloud-alibaba-dependencies` 坐标，即可享受该 BOM 内所有依赖的版本定义，无需在每个依赖中重复指定版本号。

配置文件 `application.yml` 中添加 Nacos 注册中心配置：

`application.yml`

```
1 spring:
2   application:
3     name: user-service # 服务名称
4   cloud:
5     nacos:
6       discovery:
7         server-addr: localhost:8848 # Nacos 注册中心地址
8         username: nacos # Nacos 用户名（如果启用了认证）
9         password: nacos # Nacos 密码（如果启用了认证）
```

在主类上添加 `@EnableDiscoveryClient` 注解启用服务注册功能：

`UserServiceApplication.java`

```
1 @SpringBootApplication
2 @EnableDiscoveryClient
3 public class UserServiceApplication {
4     public static void main(String[] args) {
5         SpringApplication.run(UserServiceApplication.class, args);
6     }
7 }
```

**TIP**

Spring Cloud Alibaba 2023.0.x 版本后，`@EnableDiscoveryClient` 注解已不再必需，只要引入了 Nacos Discovery 依赖并正确配置，服务就会自动注册到 Nacos。但是为了代码的可读性和明确性，建议仍然保留 `@EnableDiscoveryClient` 注解。

启动应用后，访问 Nacos 控制台（<http://localhost:8080/nacos>）登录后可以看到“user-service”服务已经注册成功，并显示其实例信息（IP 地址、端口号、状态等）。

### 3. 配置服务消费者

在服务消费者项目中同样添加 Nacos Discovery 依赖，并配置 Nacos 注册中心地址。

`application.yml`

```
1 spring:
2   application:
3     name: order-service # 服务名称
4   cloud:
5     nacos:
6       discovery:
7         server-addr: localhost:8848 # Nacos 注册中心地址
8         username: nacos # Nacos 用户名（如果启用了认证）
9         password: nacos # Nacos 密码（如果启用了认证）
```

消费者想要调用“user-service”提供的接口时，通常使用 `RestTemplate` 或 `OpenFeign` 来发起 HTTP 请求，以后者为例：

## INFO

**RestTemplate** 是 Spring 提供的同步 HTTP 客户端工具类。使用 RestTemplate 调用远程服务时, 需要硬编码服务 URL (如 `http://user-service-ip:8080/api/users`), 并需要自己实现负载均衡逻辑 (如轮询、加权等)。虽然功能完整, 但在微服务架构中显得较为繁琐。

**OpenFeign** 是 Netflix 提供的声明式 HTTP 客户端框架。开发者只需按照 Java 接口规范定义调用方法, 加上 `@FeignClient` 注解指定服务名称, OpenFeign 就会在运行时自动从 Nacos 查询服务地址并发起调用。同时集成了 Spring Cloud LoadBalancer 提供负载均衡功能, 无需手动处理。

RestTemplate 适合固定地址的简单场景 (如调用第三方 API), OpenFeign 更适合微服务间相互调用的场景, 因为它充分利用了服务注册与发现的能力。在 Spring Cloud 微服务项目中, 推荐优先使用 OpenFeign。

首先在 `pom.xml` 中添加 OpenFeign 依赖:

`pom.xml`

```
1 <dependency>
2   <groupId>org.springframework.cloud</groupId>
3   <artifactId>spring-cloud-starter-openfeign</artifactId>
4 </dependency>
```

定义 Feign 客户端接口, 用于调用 “user-service” 提供的服务:

`UserServiceClient.java`

```
1 @FeignClient(name = "user-service")
2 public interface UserServiceClient {
3
4     @GetMapping("/api/users/{id}")
5     User getUserById(@PathVariable("id") Long id);
6
7     @PostMapping("/api/users")
8     User createUser(@RequestBody User user);
9 }
```

在主类上添加 `@EnableFeignClients` 注解启用 Feign 客户端:

`OrderServiceApplication.java`

```
1 @SpringBootApplication
2 @EnableDiscoveryClient
3 @EnableFeignClients
4 public class OrderServiceApplication {
5     public static void main(String[] args) {
6         SpringApplication.run(OrderServiceApplication.class, args);
7     }
8 }
```

## TIP

虽然 “order-service” 作为服务消费者, 但它同样需要 `@EnableDiscoveryClient` 注解。原因是: 消费者需要向 Nacos 查询 “user-service” 的可用实例地址 (这就是 “发现”); 同时消费者本身也是一个微服务, 也应该向 Nacos 注册自己, 以便其他服务调用它。在 Spring Cloud 2023.0.x 后已自动启用, 但为了代码可读性建议显式添加。

然后在服务中注入并使用 Feign 客户端:

`OrderService.java`

```
1 @Service
2 public class OrderService {
3 }
```



```
4  @Autowired
5  private UserServiceClient userServiceClient;
6
7  public Order createOrder(Order order) {
8      // 调用 user-service 获取用户信息
9      User user = userServiceClient.getUserById(order.getUserId());
10     order.setName(user.getName());
11     // 其他业务逻辑...
12     return order;
13 }
14 }
```

当 OpenFeign 客户端发起调用时，它会：

1. 向 Nacos 注册中心查询 “user-service” 的可用实例列表。
2. 根据负载均衡策略（默认是轮询）选择一个实例的地址。
3. 使用 HTTP 协议向选定的实例发起请求，完成服务调用。

## 3 配置中心

在微服务架构中，每个服务都有自己的配置文件（如 `application.yml`）。随着服务数量的增加，传统的配置方式会带来一系列严峻的挑战：

- 配置分散：每个服务的配置都散落在其各自的代码库中。要修改一个通用配置（如数据库密码），可能需要修改几十个项目。
- 更新繁琐且风险高：修改配置后，需要重新打包、部署和启动服务，流程繁琐且耗时。任何一个小小的配置错误都可能导致服务启动失败。
- 环境管理复杂：一套代码通常需要部署在开发 (dev)、测试 (test)、生产 (prod) 等多个环境中。管理这些不同环境的配置文件非常容易出错。
- 安全隐患：将数据库连接、密钥等敏感信息直接放在代码库中，存在泄露风险。
- 无法动态更新：服务在运行期间无法获取最新的配置，灵活性差。例如，无法在不重启服务的情况下动态调整日志级别或开关某个功能。

统一配置管理正是为了解决这些问题而生。它将所有服务的配置信息从应用程序中抽离出来，存放在一个中心的、外部的 **配置中心 (Configuration Center)** 进行统一管理。

一个配置中心主要提供以下能力：

1. 集中管理：提供一个中心化的存储库来管理所有应用、所有环境的配置。
2. 版本控制：能够对配置的修改进行版本管理和追溯。
3. 环境隔离：轻松管理不同环境（dev/test/prod）的配置。
4. 动态刷新 (Dynamic Refresh)：在不重启服务的情况下，将配置的变更实时推送给正在运行的服务实例。这是配置中心最有价值的功能之一。
5. 权限控制：对配置的修改和读取进行精细的权限管理，保证安全。

### 3.1 主流实现

Spring Cloud 生态中主要有两个主流的配置中心解决方案：

**Spring Cloud Config** 由 Spring 官方提供，后端通常使用 Git（也支持 SVN、本地文件系统），与代码仓库结合紧密、版本管理清晰。其动态刷新通常需要结合 Spring Cloud Bus（如 RabbitMQ、Kafka）来广播刷新事件，因此整体架构相对更重，且默认没有内置可视化控制台。综合来看：方案成熟可用，但在新项目中不再是首选。

**Nacos Config** 由 Alibaba 提供，配置数据存储存储在 Nacos Server（可外挂 MySQL），并通过长轮询（Long Polling）+ 推送机制原生支持动态刷新。它与服务注册发现一体化，支持 Namespace、Group、Data ID 维度管理，同时提供可视化控制台和版本、灰度发布等能力。综合能力更完整，在 Spring Cloud Alibaba 体系中属于强烈推荐方案。

Nacos 配置中心工作流程如图 (12.2) 所示：

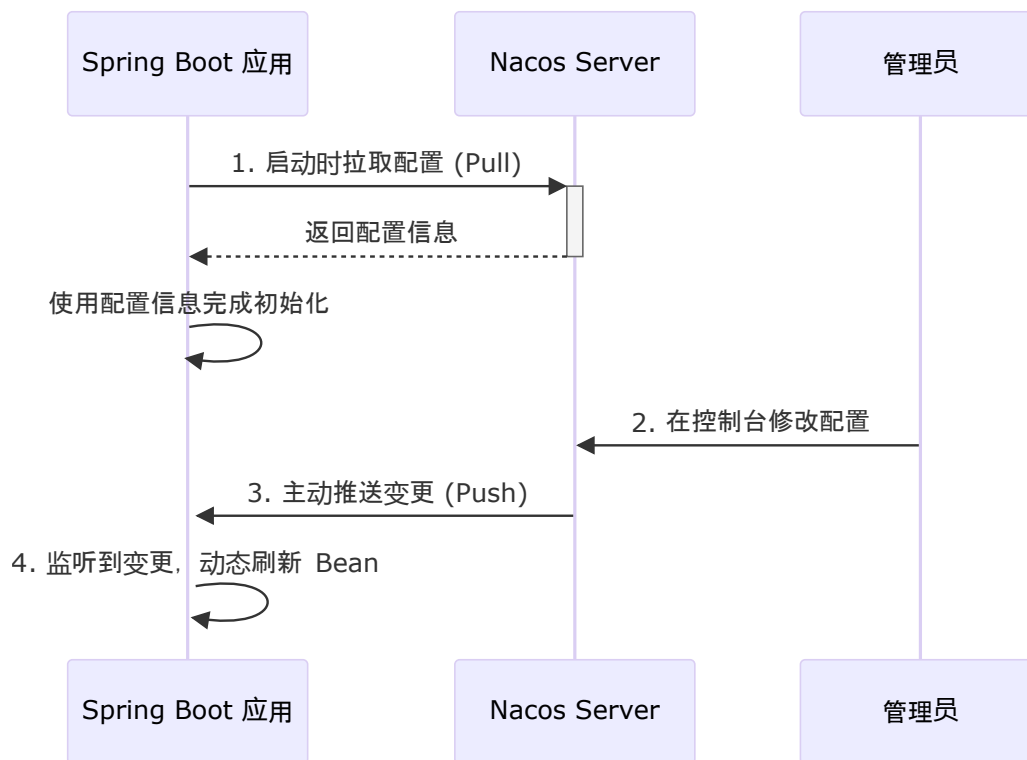


Figure 12.2: Nacos 配置中心工作流程示意图

## 3.2 Nacos 配置中心实践示例

下面以目前最流行的 Nacos 为例，展示如何实现统一配置管理。

### 1. 创建配置

启动 Nacos 服务注册中心后，访问控制台（<http://localhost:8080/nacos>）登录后进入“配置管理”页面，点击“发布配置”按钮创建一个新的配置项：

- Data ID: 配置的唯一标识符。命名规则通常是 `${spring.application.name}-${spring.profiles.active}.${file-extension}`。  
例如：`order-service-dev.yml` 表示“order-service”服务在“dev”环境下的 YAML 格式配置。
- Group: 配置组，用于隔离不同的项目或业务。默认为 `DEFAULT_GROUP`。
- 配置格式: 选择 `YAML` 或 `Properties`。
- 配置内容: 写入你的配置。

`order-service-dev.yml`

```

1 server:
2   port: 8081
3 spring:
4   datasource:
5     url: jdbc:mysql://localhost:3306/order_db
6     username: root
  
```

```
7 password: password
```

## 2. SpringBoot 客户端接入

在 SpringBoot 项目的 `pom.xml` 中添加 Nacos Config 依赖:

```
pom.xml
1 <dependency>
2   <groupId>com.alibaba.cloud</groupId>
3   <artifactId>spring-cloud-starter-alibaba-nacos-config</artifactId>
4 </dependency>
```

创建配置文件 `bootstrap.yml`, 配置 Nacos 连接信息:

```
bootstrap.yml
1 spring:
2   application:
3     name: order-service # 应用名, 用于匹配 Nacos 中的 Data ID
4   profiles:
5     active: dev # 环境标识, 用于匹配 Nacos 中的 Data ID
6   cloud:
7     nacos:
8       config:
9         server-addr: 127.0.0.1:8848 # Nacos Server 地址
10        file-extension: yaml # 指定配置文件格式
11        # group: DEFAULT_GROUP # 如果使用了非默认组, 需要指定
```

### CAUTION

对于 Spring Boot 2.4+ / Spring Cloud 2021+, 更推荐在 `application.yml` 中使用 `spring.config.import: nacos:...` 的新配置加载方式。若你仍使用 `bootstrap.yml`, 通常需要额外引入 `spring-cloud-starter-bootstrap` 才能启用旧的 bootstrap 上下文机制。

根据以上配置, Spring Boot 会去 Nacos 中寻找 Data ID 为 `order-service-dev.yaml` 的配置。

启动应用后, 应用会自动从 Nacos 加载配置并应用到 Spring 环境中。你可以在控制台看到加载日志, 确认配置已成功加载。

之后的使用和传统的 Spring Boot 配置完全一样, 你可以通过 `@Value` 注解或 `@ConfigurationProperties` 类来访问这些配置值。

### 动态刷新

当修改 Nacos 中的配置后, 客户端会通过长轮询感知变更并更新本地配置上下文, 因此应用**无需重启即可获取新配置**。

但要注意, “获取到新配置”与“业务 Bean 立即使用新值”不是同一件事:

- 配置层 (Environment) 通常会自动更新。
- Bean 层是否立即生效取决于注入方式: 使用 `Environment.getProperty(...)` 动态读取时通常可直接生效; 使用 `@Value` 注入到普通单例 Bean 时, 通常需要配合 `@RefreshScope` 才能在运行期拿到新值。
- 对 `@ConfigurationProperties` 而言, 不同版本与接入方式下重绑定行为可能不同, 建议结合当前项目版本实测验证。

**TIP**

实践建议：将“Nacos 自动刷新”理解为[配置源已更新](#)；而“业务代码实时生效”需要根据 Bean 的绑定方式决定是否引入 `@RefreshScope`。

使用 `@RefreshScope` 注解会为这个 Bean 创建一个代理（通常用在 Controller 或 Service 中）。当 Nacos 配置发生变化时，Spring Cloud 会销毁这个 Bean 的旧实例，然后根据最新的配置信息创建一个新实例。下次访问时，注入的就是新实例，从而获取到最新的配置值：

DynamicConfigBean.java

```

1 @RefreshScope
2 @Component
3 public class DynamicConfigBean {
4
5     @Value("${some.config.value}")
6     private String configValue;
7
8     public String getConfigValue() {
9         return configValue;
10    }
11 }

```

## 共享配置

在微服务中，经常有多个服务需要共享一些通用配置（如数据库连接、Redis 配置）。Nacos 提供了简单的方式来实现。可以在 `bootstrap.yml` 中配置 `shared-configs` 或 `extension-configs`。

bootstrap.yml

```

1 spring:
2   cloud:
3     nacos:
4       config:
5         server-addr: 127.0.0.1:8848
6         file-extension: yaml
7         # 共享配置
8         shared-configs:
9           - data-id: common-db.yml # 共享的数据源配置
10            refresh: true # 当共享配置变化时，也动态刷新
11           - data-id: common-redis.yml
12             refresh: true
13         # 扩展配置（优先级高于共享配置）
14         extension-configs:
15           - data-id: order-service-extension.yml # order-service 的扩展配置
16             refresh: true

```

**TIP**

共享配置的 Data ID 可以是任意命名，但建议使用统一的命名规范（如 `common-*.yaml`）来区分不同类型的共享配置。共享配置可以被多个服务同时引用，修改后会同时影响所有引用它们的服务。

**NOTE**

这里有两层优先级：

第 1 层（Nacos 内部）：[服务特定配置](#) > [扩展配置](#) > [共享配置](#) > [默认配置](#)。这一层只用于决定“多个 Nacos 配置之间”同名属性谁生效。

第 2 层（应用整体属性源，常见场景）：[启动参数](#) > [Nacos 配置](#) > [本地 application.yml](#)。这一层用于决定“启动参数、本地文件、远程配置中心”之间同名属性谁最终生效。

## 4 API 网关

在微服务架构中，系统被拆分为许多独立的服务。如果让客户端（如网页、移动 App）直接与几十甚至上百个微服务通信，会带来一系列问题：

- 客户端复杂性：客户端需要知道每个微服务的地址，并分别管理与它们的通信，逻辑非常复杂。
- 认证和授权冗余：每个微服务都需要独立实现用户认证、权限校验、安全防护等逻辑，造成大量重复工作。
- 跨域问题：如果前端应用和后端服务部署在不同域，每个服务都需要处理跨域请求 (CORS)。
- 协议转换困难：如果某些内部服务使用 gRPC 或其他协议，而客户端只支持 HTTP，直接通信会很困难。
- 服务重构困难：一旦后端对微服务进行拆分或合并，所有客户端都需要修改代码以适应新的服务地址。
- 难以统一管理：无法对整个系统的流量进行统一的监控、日志记录和限流。

API 网关 就是为了解决这些问题而诞生的。它作为整个系统的**唯一入口 (Single Point of Entry)**，像一个“前台接待”或“总机”，统一接收所有外部请求，然后根据规则将请求路由到正确的后端微服务。

Spring Cloud 生态中主要有两个主流的网关实现：

**Zuul 1.x (Netflix)** 基于 Servlet 体系的阻塞式 I/O (Blocking I/O)，实现方式以传统 Filter 链为主，配置能力相对基础。在中高并发场景下，吞吐与资源利用率通常不如非阻塞网关；同时该方案已进入维护模式，Spring 官方也不再推荐用于新项目。

**Spring Cloud Gateway (Spring 官方团队)** 基于 Spring WebFlux/Reactor 的非阻塞式 I/O (Non-blocking I/O)，在高并发与高吞吐场景下表现更优。内置了更丰富的 Route Predicate 与 Gateway Filter，既支持属性化配置，也支持 Java Fluent API 编程式配置。在当前 Spring Cloud 体系中，它是官方主推且更具长期演进价值的网关方案。

**Shenyu (Apache)** 由 Apache 基金会孵化的开源 API 网关，支持多种协议 (HTTP、gRPC、WebSocket 等)，功能丰富且高度可扩展。适合对协议支持和定制化需求较高的场景，但相对于 Spring Cloud Gateway 来说，学习曲线更陡峭，社区生态也较小。

### INFO

“ShenYu (神禹) is the honorific name of Chinese ancient monarch Xia Yu (also known in later times as Da Yu), who left behind the touching story of the three times he crossed the Yellow River for the benefit of the people and successfully managed the flooding of the river. He is known as one of the three greatest kings of ancient China, along with Yao and Shun.”

### TIP

选型建议：存量系统若已深度使用 Zuul 1.x，可在稳定优先前提下逐步迁移；新项目应优先采用 Spring Cloud Gateway，以获得更好的性能上限、扩展能力与社区支持；ShenYu 适合特定协议需求或对网关功能有高度定制化需求的场景。

API 网关通常负责处理所有与业务逻辑无关的**横切关注点 (Cross-Cutting Concerns)**：

1. 请求路由 (Routing)：核心功能。根据请求的路径、域名、请求头等信息，将请求转发到指定的后端服务。
2. 认证与授权 (Authentication & Authorization)：在网关层统一进行身份验证（如校验 JWT Token），只有合法的请求才能进入后端系统。
3. 负载均衡 (Load Balancing)：网关与服务发现组件（如 Nacos）集成，当一个服务有多个实例时，能自动将请求分发到其中一个实例上。
4. 限流与熔断 (Rate Limiting & Circuit Breaking)：保护后端服务不被突发流量冲垮。可以对指定 API



设置访问频率限制，并在后端服务不可用时快速失败（熔断）。

5. 日志与监控 (Logging & Monitoring): 集中记录所有入口流量的日志，便于审计和问题排查。
6. 协议转换 (Protocol Translation): 将外部的 HTTP 请求转换为内部服务使用的其他协议（如 gRPC）。
7. 请求/响应转换 (Request/Response Transformation): 在将请求转发给后端或将响应返回给客户端之前，修改请求头、请求体或响应体。

## 4.1 SpringCloud Gateway

Spring Cloud Gateway 是构建在 Spring WebFlux 和 Project Reactor 之上的，这使得它天然具备了非阻塞和响应式的能力。它的核心概念包括：

**Route (路由)** 定义了从请求到后端服务的映射关系，是网关最基本的构建块。它由一个 ID、一个目标 URI、一组断言 (Predicate) 和一组过滤器 (Filter) 组成。如果所有断言都为 `true`，则该路由匹配成功，请求将被转发到指定的 URI。

**Predicate (断言)** 这是一个 Java 8 的 Predicate。它的作用是匹配条件，决定一个请求是否应该被此路由处理。Gateway 内置了丰富的 Predicate 工厂，可以匹配 HTTP 请求的任何内容，例如：

- Path (路径匹配)：根据请求的 URL 路径进行匹配。
- Method (HTTP 方法匹配)：根据请求的 HTTP 方法 (GET、POST 等) 进行匹配。
- Header (请求头匹配)：根据请求头的名称和值进行匹配。
- Query (查询参数匹配)：根据 URL 中的查询参数进行匹配。
- Host (主机匹配)：根据请求的 Host 头进行匹配。
- After/Before/Between (时间匹配)：根据请求的时间进行匹配，例如在某个时间段内生效。

**Filter (过滤器)** 在请求被路由到后端服务之前或响应返回给客户端之前，对请求或响应进行修改的组件。分为全局过滤器和路由过滤器两种。常见功能为：添加/修改请求头、请求体，添加/修改响应头、响应体，记录日志，进行认证授权，限流等。

当一个请求到达 Spring Cloud Gateway 时，它的处理流程如下（如图 12.3 所示）：

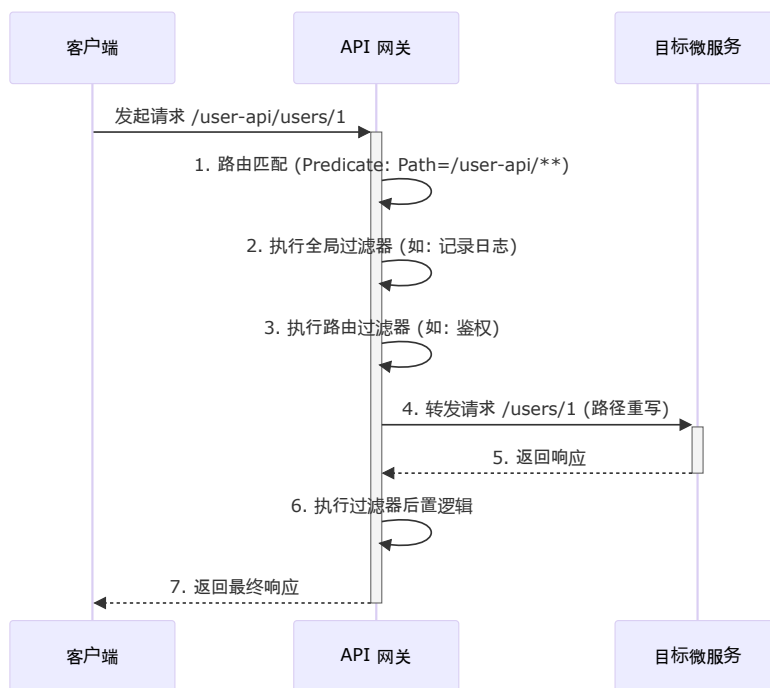


Figure 12.3: Spring Cloud Gateway 工作流程

1. Gateway Handler Mapping: 根据请求信息（路径、方法等）和配置的 Predicate，找到匹配的路由。

2. Gateway Web Handler: 将请求传递给一个特定的过滤器链 (Filter Chain)。
3. 执行过滤器链:
  - 请求会先经过所有 GlobalFilter 的 pre 阶段 (前置处理)。
  - 然后经过该路由特定的 GatewayFilter 的 pre 阶段。
  - 最后, 代理请求被发送到目标服务。
4. 服务响应: 目标服务返回响应。
5. 执行过滤器链 (反向):
  - 响应会反向经过 GatewayFilter 的 post 阶段 (后置处理)。
  - 再反向经过 GlobalFilter 的 post 阶段。
6. 返回响应: 最终将处理过的响应返回给客户端。

## 4.2 SpringCloud Gateway 实践示例

下面以 Spring Cloud Gateway 为例, 展示如何实现一个简单的 API 网关。

假设我们有两个后端服务: “user-service” 和 “order-service”, 它们分别运行在不同的端口上。我们希望通过 API 网关将它们暴露给客户端。

### 1. 添加依赖

在 API 网关项目的 pom.xml 中添加 Spring Cloud Gateway 依赖:

```
pom.xml
1 <dependency>
2   <groupId>org.springframework.cloud</groupId>
3   <artifactId>spring-cloud-starter-gateway</artifactId>
4 </dependency>
5 <!-- 集成 Nacos 服务发现 -->
6 <dependency>
7   <groupId>com.alibaba.cloud</groupId>
8   <artifactId>spring-cloud-starter-alibaba-nacos-discovery</artifactId>
9 </dependency>
10 <!-- 使用 lb:// 路由时建议显式引入 LoadBalancer -->
11 <dependency>
12   <groupId>org.springframework.cloud</groupId>
13   <artifactId>spring-cloud-starter-loadbalancer</artifactId>
14 </dependency>
```

### 2. 配置网关模块配置文件

在 application.yml 中配置 Spring Cloud Gateway 和 Nacos 服务发现:

```
application.yml
1 server:
2   port: 8888 # 网关端口
3
4 spring:
5   application:
6     name: api-gateway
7   cloud:
8     nacos:
9       discovery:
10        server-addr: 127.0.0.1:8848 # Nacos 地址
11
12    gateway:
13      routes:
14        # 路由规则1: 访问用户服务
15        - id: user_service_route # 路由的唯一ID
16          uri: lb://user-service
```

```

17     # Predicates (断言/匹配条件), 必须所有条件都满足
18     predicates:
19     # 当请求路径匹配 /user-api/** 时, 此路由生效
20     - Path=/user-api/**
21     # Filters (过滤器), 对请求进行处理
22     filters:
23     # StripPrefix=1 会去掉路径中的第一个部分
24     # 例如: /user-api/users/1 -> /users/1
25     - StripPrefix=1
26     # 也可以添加请求头
27     - AddRequestHeader=X-Request-From, api-gateway
28
29     # 路由规则2: 访问订单服务
30     - id: order_service_route
31     uri: lb://order-service
32     predicates:
33     - Path=/order-api/**
34     filters:
35     - StripPrefix=1

```

**TIP**

在路由配置中, `uri: lb://user-service` 表示使用负载均衡 (Load Balancing) 方式调用服务注册中心中名为 “user-service” 的服务实例。这样, 当 “user-service” 有多个实例时, 网关会自动将请求分发到其中一个实例上, 实现了负载均衡。

本示例使用**显式路由** () 以便精确控制对外暴露的 API。仅在需要按服务名自动暴露全部路由时, 再考虑开启 `spring.cloud.gateway.discovery.locator.enabled=true`。

通过以上配置:

1. 当客户端访问 `http://localhost:8888/user-api/users/123` 时, 网关会匹配到 `user_service_route`。
2. `StripPrefix=1` 过滤器会将路径重写为 `/users/123`。
3. 网关通过 Nacos 找到 `user-service` 的一个健康实例 (例如 `http://192.168.1.10:8081`)。
4. 最终将请求 `GET /users/123` 发送到该实例。

## 5 负载均衡与熔断降级



# Chapter 13

## 第 10 章

---

### 1 占位内容

这里是 chap10 的初始内容。

# Chapter 14

## 第 10 章

---

### 1 占位内容

这里是 chap10 的初始内容。

# IV

Part

## Spring Boot 源码